

1. DDL (Data Definition Language) Statements

DDL is used to define, modify, and remove the descriptions (schemas) of tables in a database.

- **CREATE TABLE:** Used to define a new base relation by specifying its name, attributes, and data types (e.g., `INTEGER`, `FLOAT`, `CHAR(n)`, `VARCHAR(n)`).
 - *Example:* `CREATE TABLE DEPARTMENT (DNAME VARCHAR(10) NOT NULL, DNUMBER INTEGER NOT NULL, MGRSSN CHAR(9));`
- **ALTER TABLE:** Used to add or drop attributes, change column definitions, or add/drop constraints in an existing table.
 - *Example (Add Column):* `ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);`
 - *Example (Drop Column):* `ALTER TABLE EMPLOYEE DROP COLUMN Address CASCADE;`
- **DROP TABLE:** Used to remove a relation and its definition from the database.
 - *Example:* `DROP TABLE DEPENDENT CASCADE;`

2. Specifying Primary Key and Foreign Key Constraints

These are specified within the `CREATE TABLE` command to ensure data integrity.

- **Primary Key:** Uniquely identifies each tuple in a relation.
 - *Syntax:* `PRIMARY KEY (attribute_name).`
- **Foreign Key:** A set of attributes in one table that refers to the primary key of another table, creating a referential integrity link.
 - *Syntax:* `FOREIGN KEY (attribute) REFERENCES table_name(attribute).`
 - *Options:* You can specify actions like `ON DELETE CASCADE`, `ON DELETE SET NULL`, or `ON UPDATE CASCADE`.

3. Check Constraints

The **CHECK** clause is used to restrict attribute or domain values based on a specific condition.

- It must hold true for every state of the database.
- *Example:* To restrict department numbers to between 1 and 20: `Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);`
- It can also be applied to a domain: `CREATE DOMAIN DN AS INTEGER CHECK (DN > 0 AND DN < 21);`

4. SQL Statements for Different Databases

The sources provide queries primarily for the **Employee (Company)** and **Hotel/Guest** databases.

- **Employee Database:**

- Retrieve birth date and address of 'John B. Smith': `SELECT BDATE, ADDRESS FROM EMPLOYEE WHERE FNAME='John' AND LNAME='Smith';`.
- Retrieve names of employees in the 'Research' department: `SELECT FNAME, LNAME FROM EMPLOYEE, DEPARTMENT WHERE DNAME='Research' AND DNUMBER=DNO;`.
- **Hotel/Guest Database:**
 - List guests in 'Chandigarh' ordered by name: `SELECT name, address FROM Guest WHERE address LIKE '%Chandigarh%' ORDER BY name;`.
- **Student Database:** Schema includes **STUDENT**(Name, Student_number, Class, Major).
- **Hospital Database:** Not explicitly detailed with SQL examples in the provided snippets.

5. SQL Clauses: EXISTS, IN, LIKE, OR, and Nested Queries

- **IN:** Compares a value to a set of values returned by a subquery or an explicit list.
 - *Example:* `WHERE PNO IN (1, 2, 3);`.
- **LIKE:** Used for partial string matching. % replaces multiple characters, and _ replaces one.
 - *Example:* `WHERE ADDRESS LIKE '%Houston,TX%';`.
- **EXISTS:** Checks if the result of a correlated nested query is non-empty.
 - *Example:* Find employees with dependents: `WHERE EXISTS (SELECT * FROM DEPENDENT WHERE SSN=ESSN);`.
- **OR:** Used in the `WHERE` clause to combine multiple conditions.
- **Nested Queries:** A complete `SELECT` query specified within the `WHERE` clause of another query.

6. Alter Table, Delete, and Update Commands

- **ALTER TABLE:** (See point 1).
- **DELETE:** Removes tuples that satisfy a `WHERE` condition.
 - *Example:* `DELETE FROM EMPLOYEE WHERE LNAME='Brown';`.
- **UPDATE:** Modifies attribute values in selected tuples.
 - *Example:* `UPDATE PROJECT SET PLOCATION = 'Bellaire' WHERE PNUMBER=10;`.

7. Aggregate Functions with GROUP BY and HAVING

Aggregate functions include **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**.

- **GROUP BY:** Groups tuples by one or more attributes so functions can be applied to each subgroup.
- **HAVING:** Applies a selection condition to the groups themselves (rather than individual tuples).
- *Example:* Find projects with more than 2 employees: `SELECT PNUMBER, COUNT(*) FROM PROJECT, WORKS_ON WHERE PNUMBER=PNO GROUP BY PNUMBER HAVING COUNT(*) > 2;`.

8. Trigger Commands

A trigger specifies automatic actions the DBMS performs when certain events occur.

- **Components:** Event (e.g., INSERT), Condition (WHEN clause), and Action (SQL statement or Procedure).
- **Types:** Can be specified to run `BEFORE` or `AFTER` the triggering event.
- *Example Event:* Inserting a new employee or changing a salary.

9. Creation of Views

A view is a virtual table derived from other base tables or views.

- **Syntax:** `CREATE VIEW view_name AS SELECT ....`
- **Example:** `CREATE VIEW WORKS_ON1 AS SELECT Fname, Lname, Pname FROM EMPLOYEE, PROJECT, WORKS_ON WHERE Ssn=Essn AND Pno=Pnumber;.`
- Views can be dropped using `DROP VIEW view_name;.`

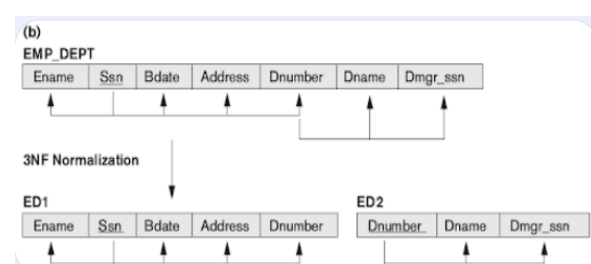
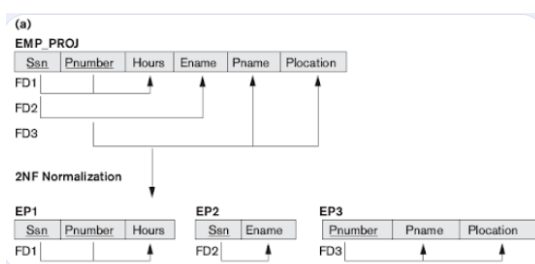
10. Example for Stored Procedures

The sources mention stored procedures as actions taken by triggers.

- **Example:** `INFORM_SUPERVISOR` is a procedure called when an employee's salary exceeds their supervisor's salary.

11. Explanation of 1NF, 2NF, and 3NF

- **First Normal Form (1NF):** Disallows multivalued and composite attributes. Every attribute must contain only atomic (indivisible) values.
 - *Example:* Moving `Dlocations` to a separate table `DEPT_LOCATIONS`.
- **Second Normal Form (2NF):** A relation is in 2NF if every nonprime attribute is **fully functionally dependent** on the primary key.
 - *Example:* Splitting a table where some attributes only depend on *part* of a composite key.
- **Third Normal Form (3NF):** A relation is in 3NF if it is in 2NF and has no **transitive dependencies** (where a nonprime attribute depends on another nonprime attribute).



12. Guidelines of Database Design

1. **Clear Semantics:** Attributes in a relation should have a clear real-world meaning; don't mix entities.
2. **Avoid Redundancy:** Design schemas to minimize redundant information and avoid update anomalies.
3. **Minimize NULLs:** Avoid attributes that will frequently have NULL values.
4. **No Spurious Tuples:** Ensure relations can be joined on primary key/foreign key pairs to prevent invalid data.

13. Anomalies: Insert, Delete, and Update

Anomalies occur when redundant data is stored in natural joins of base relations.

- **Insert Anomaly:** Cannot insert a new department if it has no employees yet (if SSN is the primary key).
- **Delete Anomaly:** Deleting the last employee of a department accidentally deletes all information about that department.
- **Update (Modification) Anomaly:** Changing one department attribute requires updating many employee tuples to keep the database consistent.

14. How to Avoid Anomalies?

Anomalies are avoided through **Normalization**, which involves decomposing large relations with redundancies into smaller, higher-quality relations (2NF, 3NF, etc.).

15. Lossless Join (Nonadditive Join)

This property ensures that joining decomposed relations via `NATURAL JOIN` will recover exactly the same set of tuples as the original relation, without generating "spurious" (invalid) data.

	Ssn	Ename	Pnumber	Pname	Plocation	Hours
R_1	b_{11}	a_2	b_{13}	b_{14}	a_5	b_{16}
R_2	a_1	b_{22}	a_3	a_4	a_5	a_6

(No changes to matrix after applying functional dependencies)

16. Equivalence of Functional Dependencies (FDs)

Two sets of FDs, E and F, are equivalent if their closures are equal $E^+ = F^+$

- This means every FD in E can be inferred from F, and every FD in F can be inferred from E.

17. Inference Rules with Proof

Inference rules (IR1-IR6) are used to derive new FDs from a given set.

- **Reflexive Rule (IR1):** If $X \supseteq Y$, then $X \rightarrow Y$.
- **Augmentation Rule (IR2):** If $X \rightarrow Y$, then $XZ \rightarrow YZ$.
 - *Mathematical Proof:* Provided in **DBS_Unit_2_Part_4.pptx, Page 125**, demonstrating that if $XZ \rightarrow YZ$ were false, it would contradict the original dependency $X \rightarrow Y$

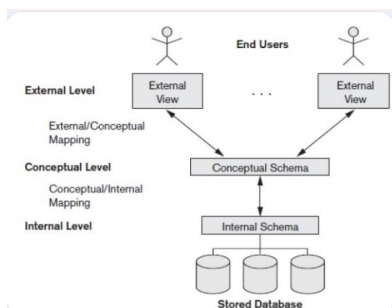
18. Canonical or Minimal Cover

A minimal cover is the most reduced set of FDs that preserves all original dependencies.

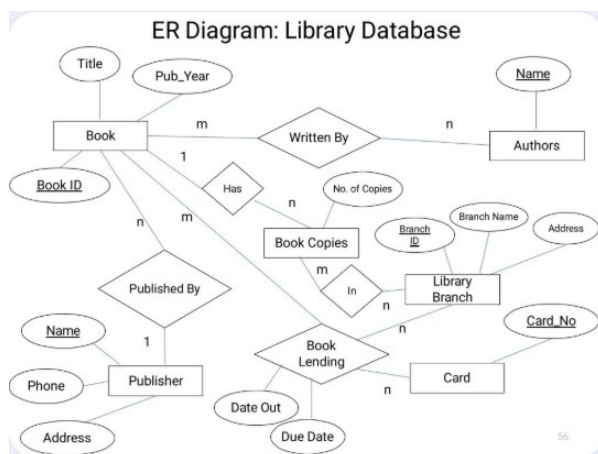
- **Algorithm:**
 1. Convert RHS of all FDs to single attributes (Canonical form).
 2. Remove extraneous attributes from the LHS.
 3. Remove redundant functional dependencies.
 - *Example:* $E: \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$ becomes $\{B \rightarrow D, D \rightarrow A\}$
-

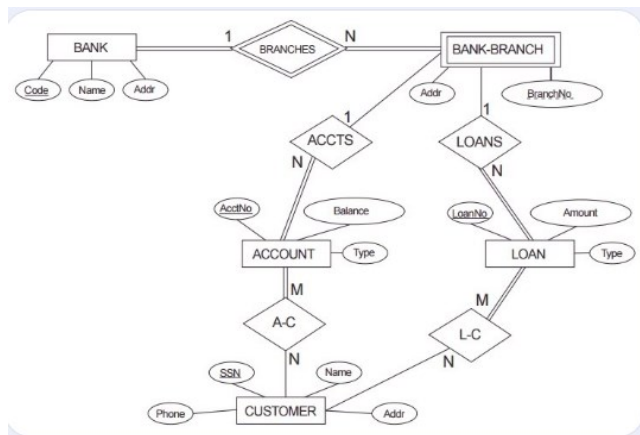
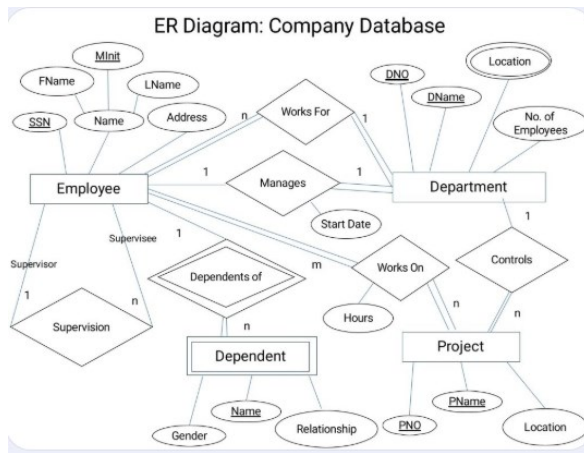
Diagram References Summary

Three-Schema Architecture:



ER Diagram





Bank ER Diagram

Functional Dependency

https://youtu.be/fZ41WtisQgo?si=chaF7X_u4OH1bEgi

Functional Dependency in DBMS

A **functional dependency (FD)** is a relationship between attributes where one attribute uniquely determines another. It is represented as $X \rightarrow Y$, meaning X determines Y. ([GeeksforGeeks](#))

Partial Dependency

<https://youtu.be/pYASiat3eQg?si=FfY0y8IIUQOHEk87>

A **partial dependency** occurs when a non-prime attribute depends on only a part of a composite key, not the whole key. This leads to redundancy and is removed in 2NF.

Types of Functional Dependency

 <https://youtu.be/2JQ0iLcFNDY?si=1wwUXunmexkOypjg>

Types include:

- **Trivial FD**
- **Non-Trivial FD**
- **Completely Non-Trivial FD**

These classify dependencies based on how attributes relate within a table.

Normalization

 <https://youtu.be/UF0UHCX-z0E?si=7LjBrWAN7zrzXcpw>

Normalization is the process of organizing data to **reduce redundancy and improve data integrity** using functional dependencies. ([GeeksforGeeks](#))

First Normal Form (1NF)

 <https://youtu.be/ew4KRX0HHXk?si=65tBmtAGc0NMQeD0>

- No repeating groups
 - All attributes contain atomic values
 - Each record is unique
-

Second Normal Form (2NF)

 https://youtu.be/cCXxKpNSXrw?si=-7_F4PTFCVCfacMw

- Must be in 1NF
 - No partial dependency
 - All non-key attributes depend on the whole primary key
-

Third Normal Form (3NF)

 https://youtu.be/ZLzO_lbFcFA?si=47qOpJpRL30U3BsX

- Must be in 2NF
 - No transitive dependency
 - Non-key attributes depend only on the primary key
-

Boyce-Codd Normal Form (BCNF)

 https://youtu.be/DfT_ngTgDws?si=MfxkPT2grPhkq_P1

- Stronger version of 3NF
 - Every determinant must be a candidate key
-

Minimal Cover

 <https://youtu.be/sS-LJMTVVj8?si=YbH1JODB0jfDHNEy>

 https://youtu.be/-DsnJygWgi4?si=QoP_dAJU38h3U4T3

A **minimal cover** is the simplified set of functional dependencies that:

- Has no redundant attributes
 - Has no redundant dependencies
 - Is equivalent to the original FD set
-